Splunk SIEM Project

Analyzing HTTP Log Files Using Splunk SIEM

Hassen Hannachi

SOC Analyst Lab Series · Blue Team

March 25, 2026

Introduction

HTTP (Hypertext Transfer Protocol) log files contain valuable information about web server activity, including requests, responses, user agents, and much more. Analyzing HTTP logs using Splunk SIEM enables security professionals to monitor web traffic, detect anomalies, and identify potential security threats in the network.

Project Overview: In this project, we upload sample HTTP log files to Splunk SIEM and perform various analyses to gain insights into web server activity within the network.

Prerequisites: Before starting this project, ensure the following are in place:

- A Splunk instance is installed and configured.
- HTTP log data sources are configured to forward logs to Splunk.

HTTP Log Field	Description
`src_ip`	Source (client) IP address
`dest_ip`	Destination (server) IP address
`src_port`	Source port
`dest_port`	Destination port
`method`	HTTP request method (GET, POST, etc.)
`status`	HTTP response status text (OK, Not Found, etc.)
`status_code`	Numeric HTTP response code (200, 404, 500, etc.)
`uri`	Requested URL or endpoint
`user_agent`	Browser or client identifier string

Part 1 — Uploading HTTP Log Files to Splunk

1. Prepare Sample HTTP Log Files

- Obtain sample HTTP log files in a suitable format (e.g., plain text or compressed `.gz` files). A public sample set is available at:
  https://www.secrepo.com/maccdc2012/http.log.gz

- Ensure the log files contain relevant HTTP events including timestamps, request methods, URLs, response codes, and user agents.
- Save the sample log files in a directory accessible by the Splunk instance.

2. Upload Log Files to Splunk

- Log in to the Splunk web interface.
- Navigate to **Settings** → **Add Data**.

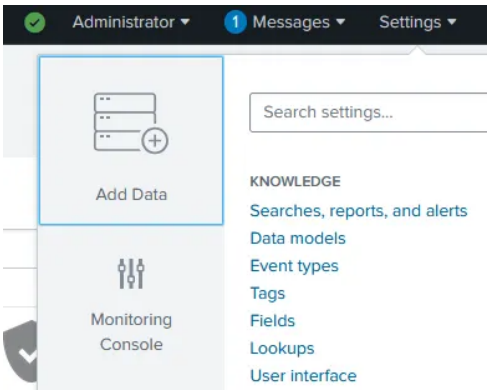

Figure 1: Splunk — Settings → Add Data menu

- Select **Upload** as the data input method.

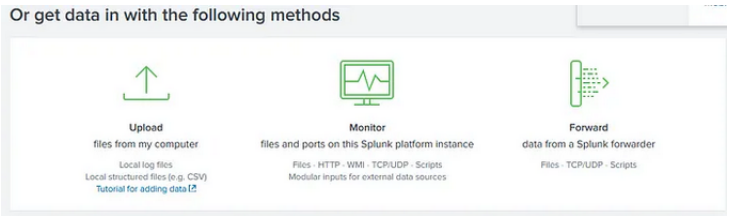

Figure 2: Selecting the Upload data input method

3. Choose the Log File

Click **Select File** and choose the sample HTTP log file prepared in Step 1.

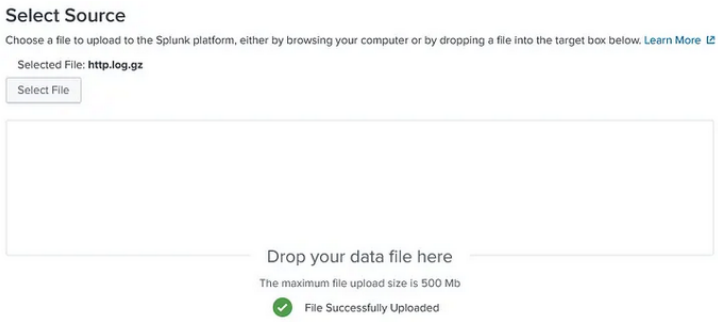

Figure 3: Selecting the sample HTTP log file for upload

4. Set the Source Type

In the **Set Source Type** section, specify the source type for the uploaded log file. Choose the appropriate source type for HTTP logs — for example, `access_combined` — or define a custom source type if needed (e.g., `httplogs`).

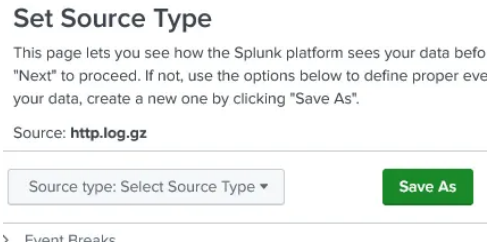

Figure 4: Setting the source type for the HTTP log file

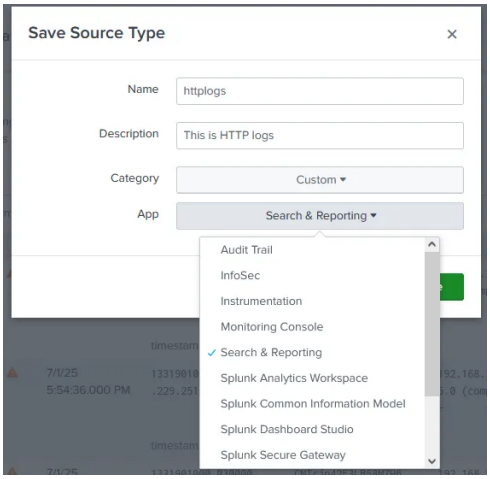

Figure 5: Confirming the selected source type settings

5. Review Settings

Review the remaining settings including **index**, **host**, and **sourcetype**. Ensure all values are configured correctly to match the sample HTTP log file.

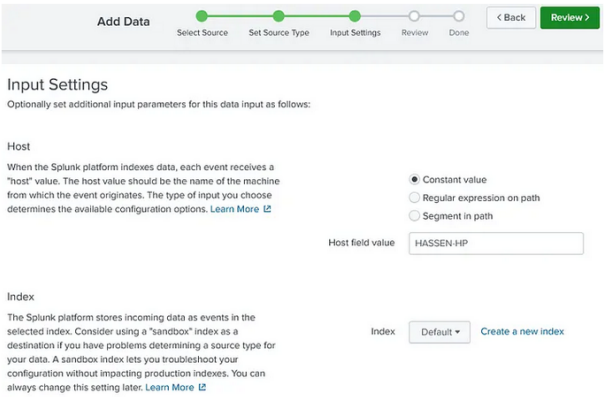

Figure 6: Reviewing index, host, and sourcetype settings

6. Submit the Upload

- Click the **Review** button and inspect all settings one final time.
- Click **Submit** to upload the HTTP log file to Splunk.

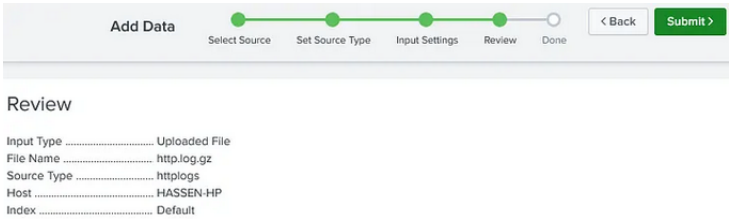

Figure 7: Submitting the HTTP log file upload

7. Verify the Upload

After uploading, navigate to the search bar (**Start Searching**) and run the following query to confirm that the uploaded HTTP events are visible:

```
source="http.log.gz" host="HASSEN-HP" sourcetype="httplogs"
```

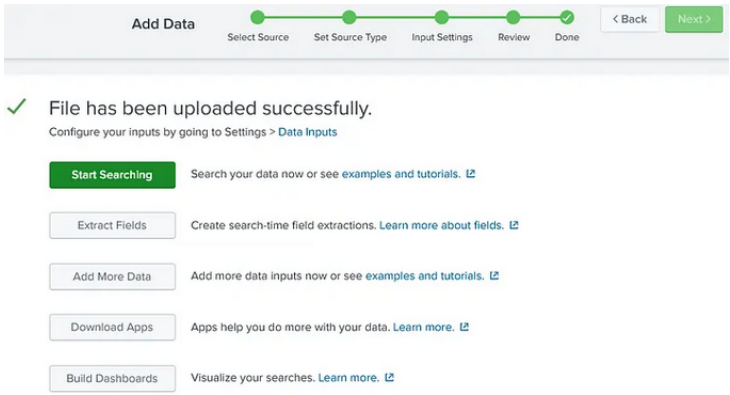

Figure 8: Navigating to the Splunk search bar after upload

8. Parse and Extract Fields

Navigate to **Extract New Fields** in the Splunk interface to define custom field extractions from the raw log data.

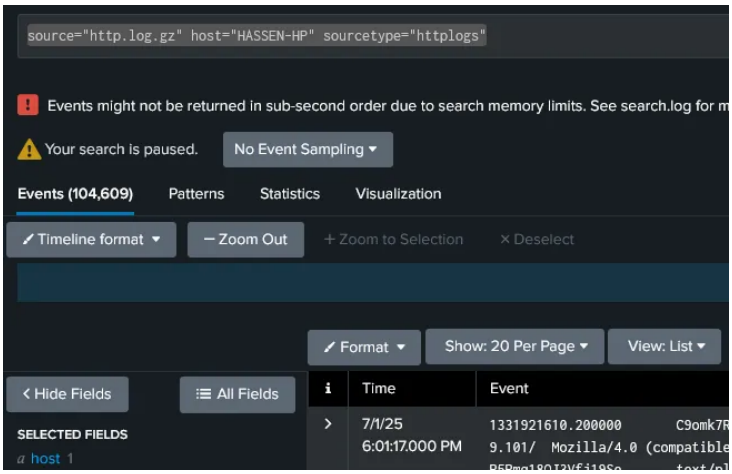

Figure 9: Verifying that uploaded HTTP events appear in search results

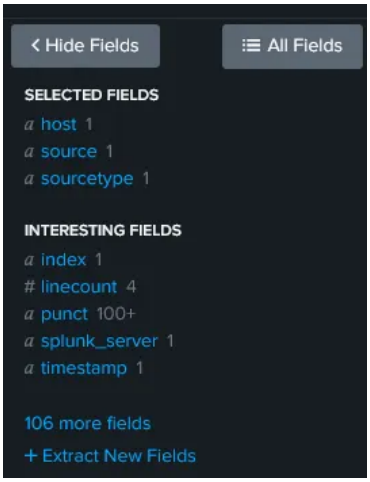

Figure 10: Opening the Extract New Fields panel

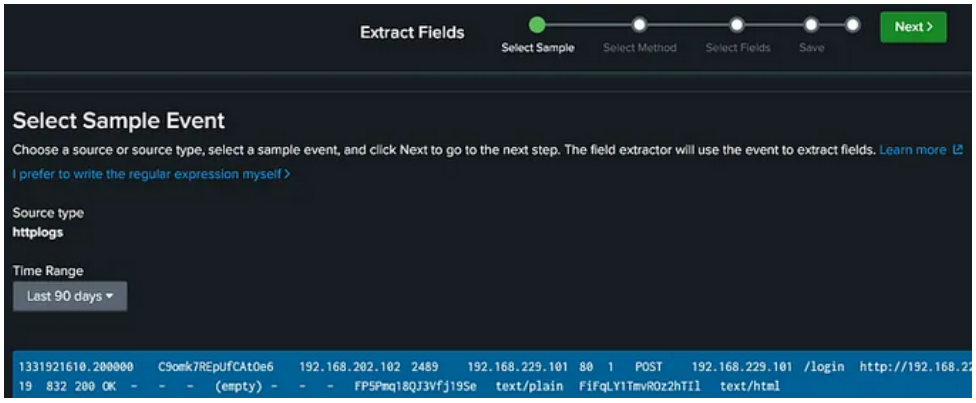

Figure 11: Selecting a sample event for field extraction

Select any event from the list and click **Next**.

Select **Regular Expression** as the extraction method (Delimiters can also be used), then click **Next**.

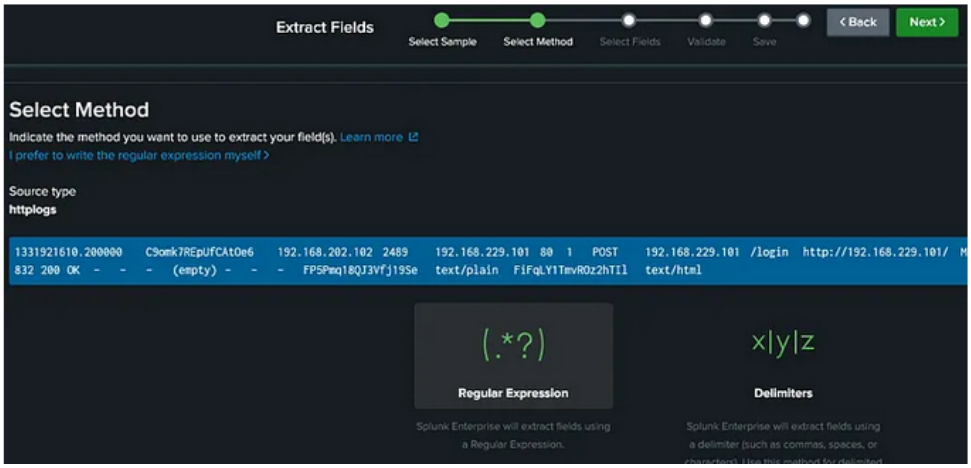

Figure 12: Choosing Regular Expression as the extraction method

Extract the following fields one by one by clicking on the relevant value in the event preview and assigning a field name:

Value Selected	Field Name	Description
`192.168.202.83`	`src_ip`	Client machine IP address
`192.168.299.201`	`dest_ip`	Destination server IP (port 80)
`POST`	`method`	HTTP request method
`OK`	`status`	HTTP response status text

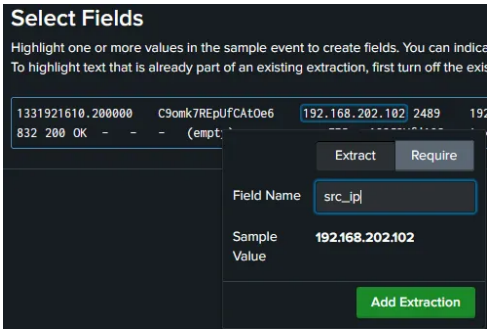

Figure 13: Extracting the src_ip field (client address)

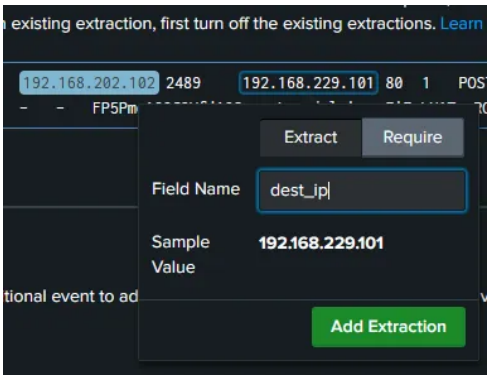

Figure 14: Extracting the dest_ip field (server address)

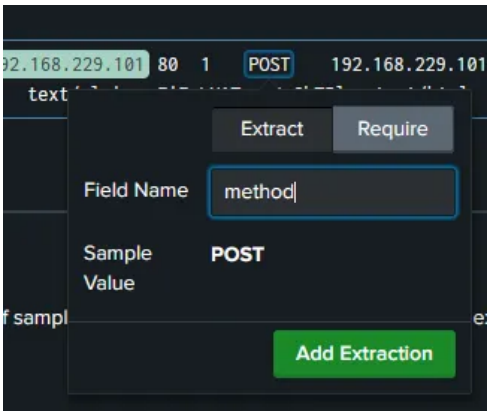

Figure 15: Extracting the method field (POST)

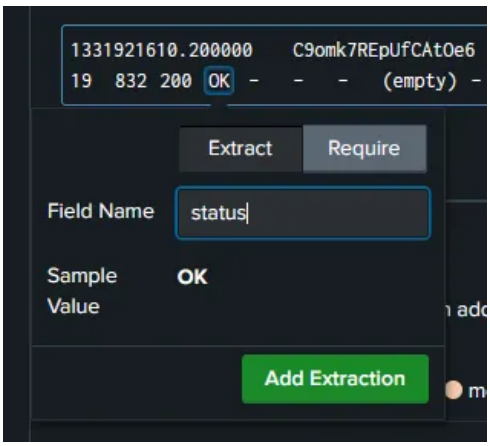

Figure 16: Extracting the status field (OK)

After all fields are validated and saved, click **Finish** to complete the extraction.

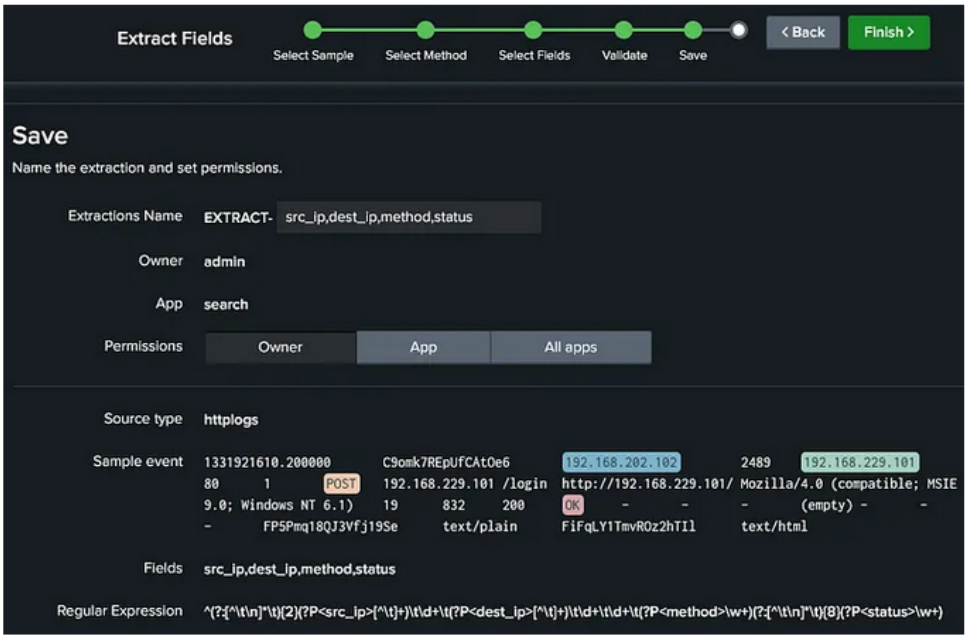

Figure 17: Validating and saving all extracted fields

Click **Explore the fields I just created in Search** to verify the results.

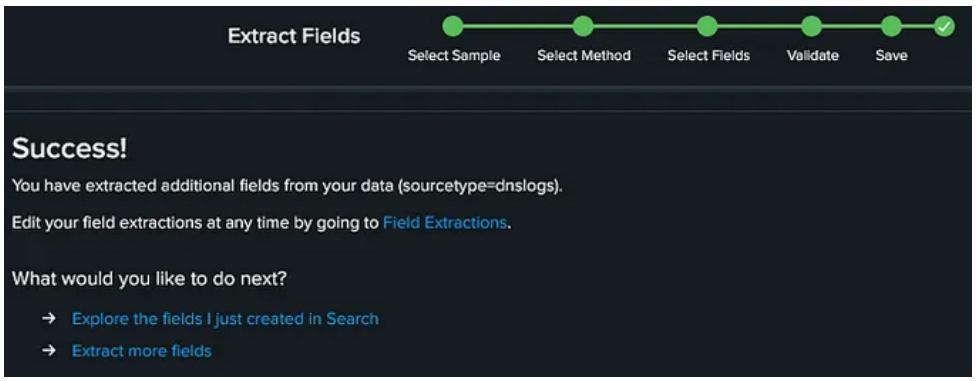

Figure 18: Navigating to search after field extraction

Result: The **Interesting Fields** panel now contains all the newly extracted fields:
`dest_ip` `method` `src_ip` `status` `src_port` `dest_port` `status_code`

Figure 19: Interesting Fields panel showing all extracted fields

Part 2 — Analyzing HTTP Log Files in Splunk

9. Search for HTTP Events

Open the Splunk search bar and enter the following query to retrieve all HTTP events from the uploaded log source:

```
index=_* OR index=* sourcetype=httplogs
```

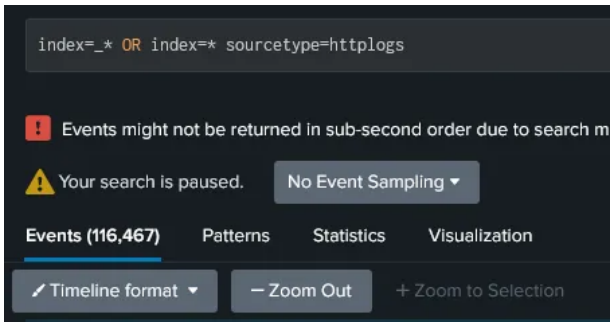

Figure 20: Retrieving all HTTP events from the Splunk index

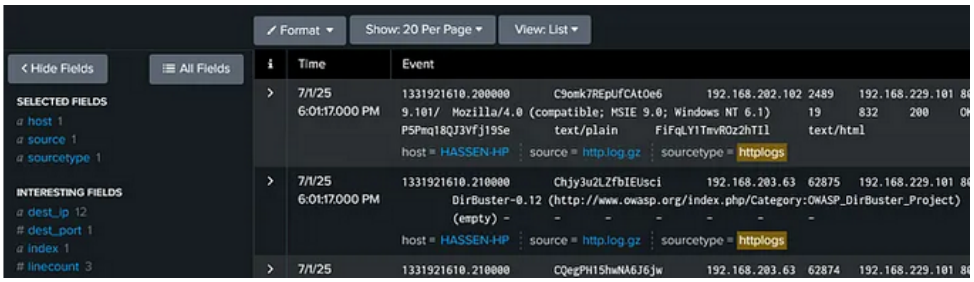

Figure 21: HTTP events displayed in the Splunk search results view

10. Extract Relevant Fields with Rex

Identify key fields in the HTTP logs such as timestamps, request methods, URLs, response codes, and user agents. Use Splunk's `rex` command with a regular expression to extract fields for richer analysis:

```
index=_* OR index=* sourcetype=httplogs
| rex field=_raw
  "(?<src_ip>\d{1,3}(?:\.\d{1,3}){3})\t
   (?<dest_ip>\d{1,3}(?:\.\d{1,3}){3})\t
   (?<status_code>\d{3})"
```

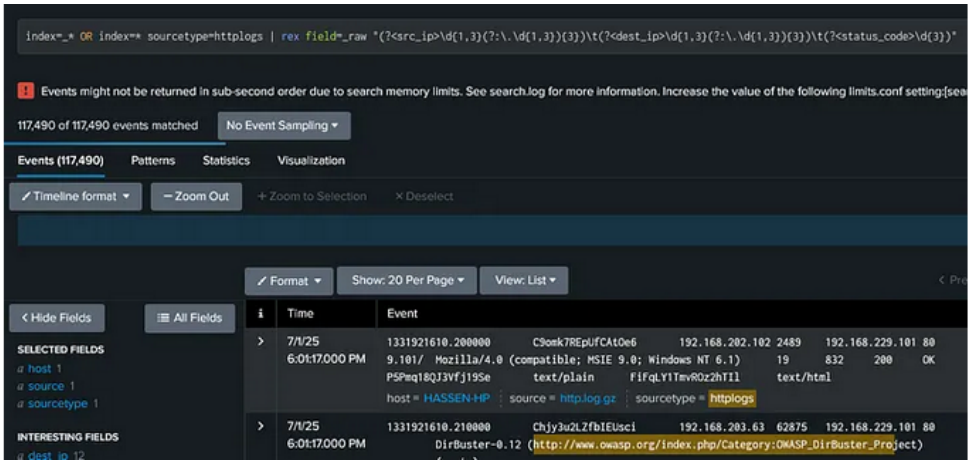

Figure 22: Using the rex command to extract src_ip, dest_ip, and status_code

11. Analyze Web Traffic Patterns

11.1 Distribution of HTTP Request Methods

Determine the distribution of request methods (GET, POST, HEAD, etc.) to understand overall web traffic patterns:

```
index=_* OR index=* sourcetype=httplogs
| stats count by method
```

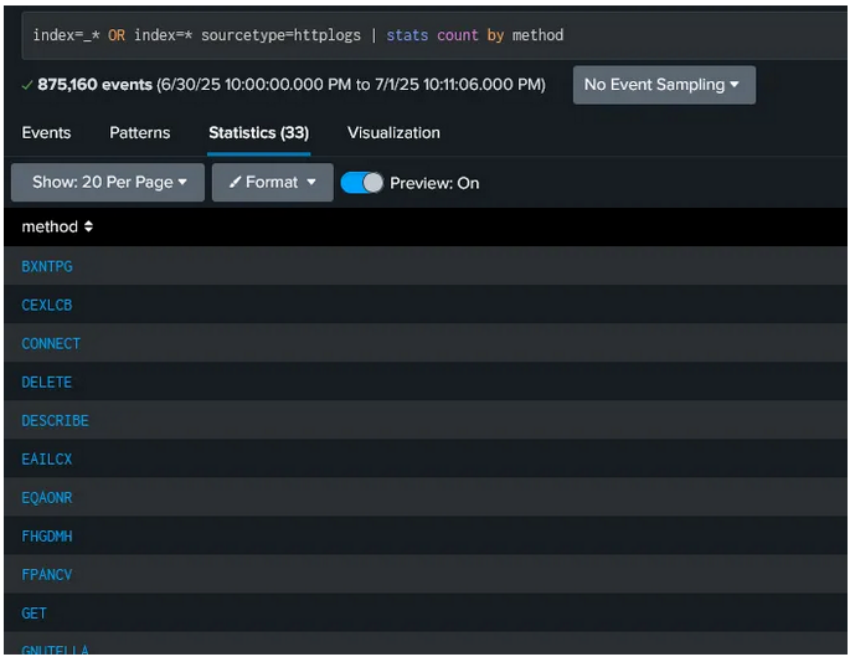

Figure 23: Count of HTTP events grouped by request method

11.2 Top Accessed URLs and Endpoints

Identify the top URLs or endpoints most frequently requested by clients:

```
index=_* OR index=* sourcetype=httplogs
| top limit=10 uri
```

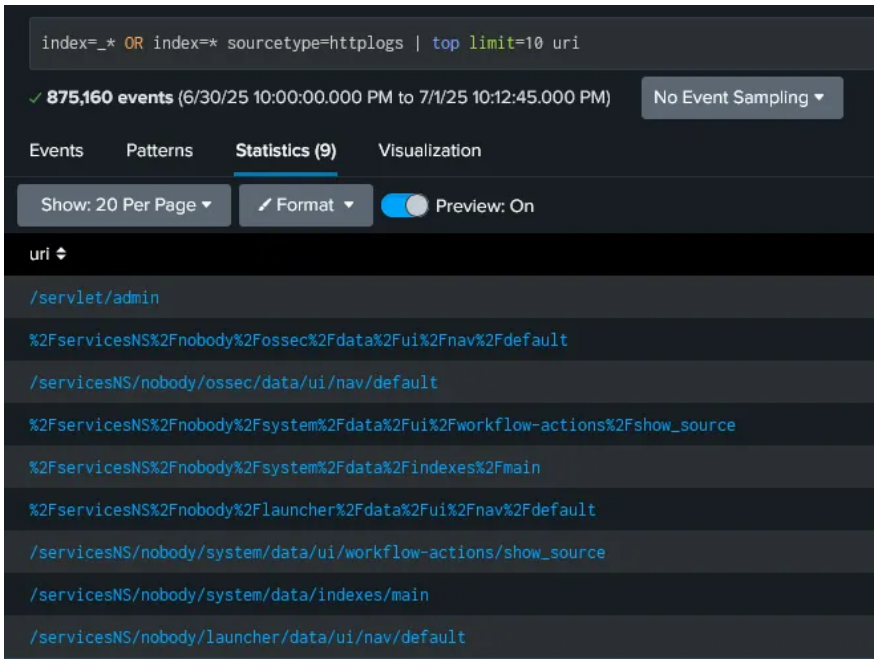

Figure 24: Top 10 most accessed URLs or endpoints

11.3 Response Code Distribution

Analyze the breakdown of HTTP response codes to distinguish successful requests from errors:

```
index=_* OR index=* sourcetype=httplogs
| stats count by status
```

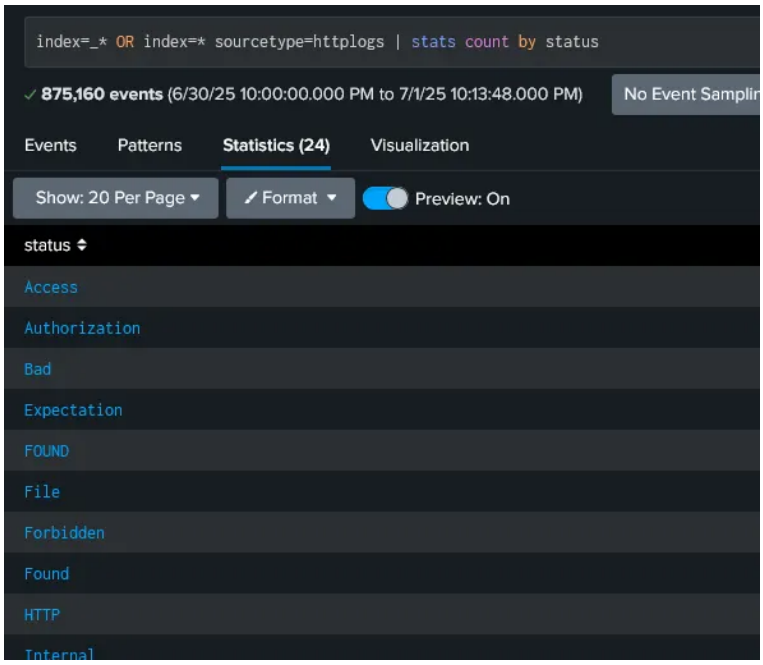

Figure 25: Count of HTTP events grouped by response status code

12. Detect Anomalies

12.1 File Transfer Activity Over Time

Look for unusual patterns in transfer activity by charting event volume over time:

```
index=_* OR index=* sourcetype=httplogs
| timechart span=4h count by status
```

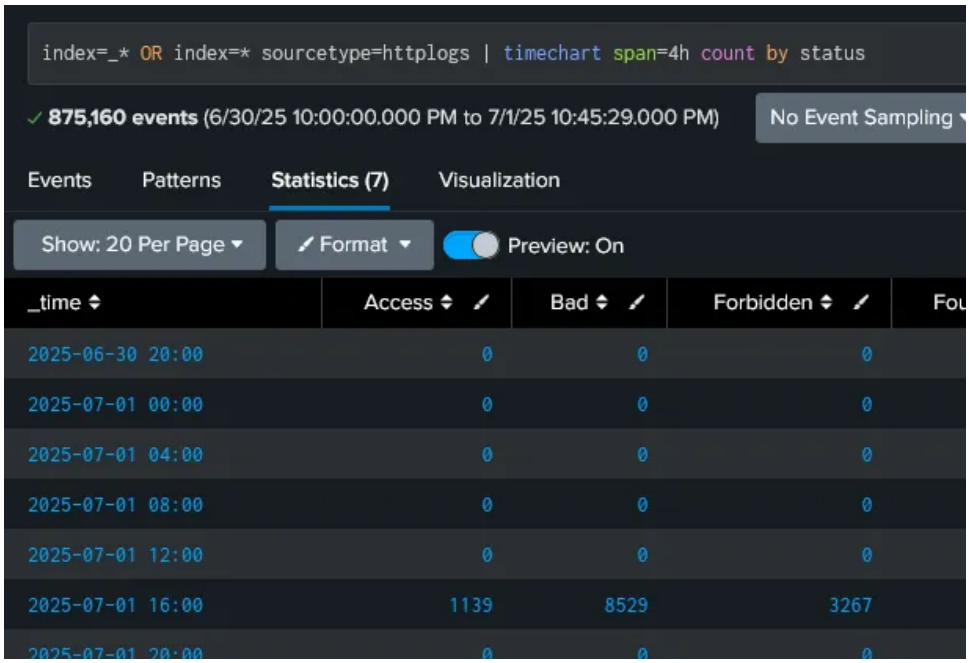

Figure 26: Time-chart of HTTP events grouped by status over 4-hour spans

12.2 High Volume of Error Responses

Analyze requests that returned HTTP error codes (4xx and 5xx) to detect potential attack activity or misconfigurations:

```
index=_* OR index=* sourcetype=httplogs
| where status >= 400
| stats count by status
```

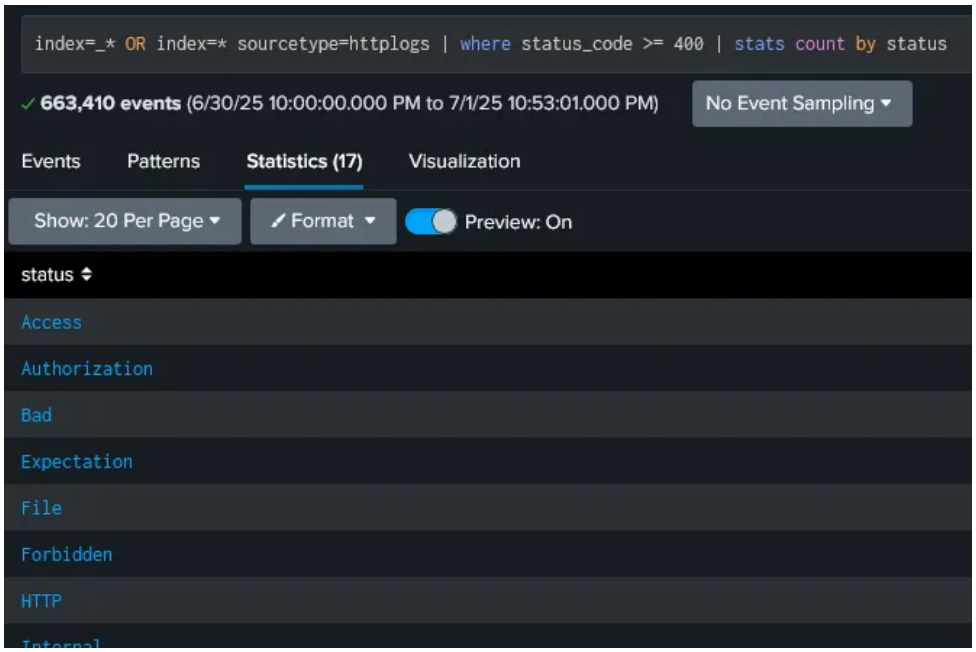

Figure 27: Count of HTTP error responses grouped by status code

12.3 Top Offending Source IPs

Investigate which source IP addresses are generating the highest number of errors, which may indicate scanning or brute-force activity:

```
index=_* OR index=* sourcetype=httplogs
| where status >= 400
| top src_ip by status_code
```

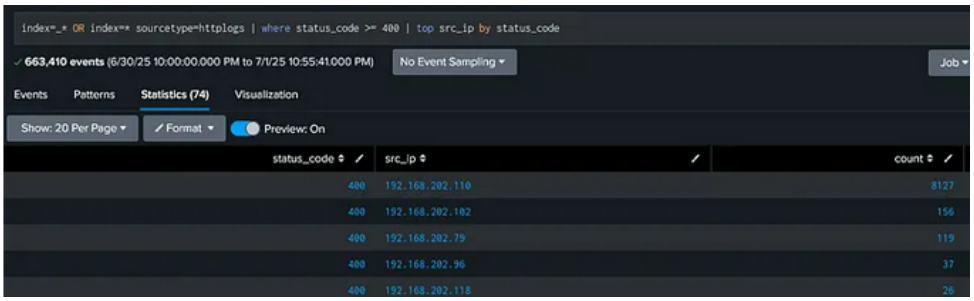

Figure 28: Top source IPs generating HTTP error responses

12.4 Top 10 Suspicious Source IPs by Transfer Volume

Identify the source IP addresses responsible for the highest number of file transfer requests:

```
index=_* OR index=* sourcetype=httplogs
| top src_ip limit=10
```

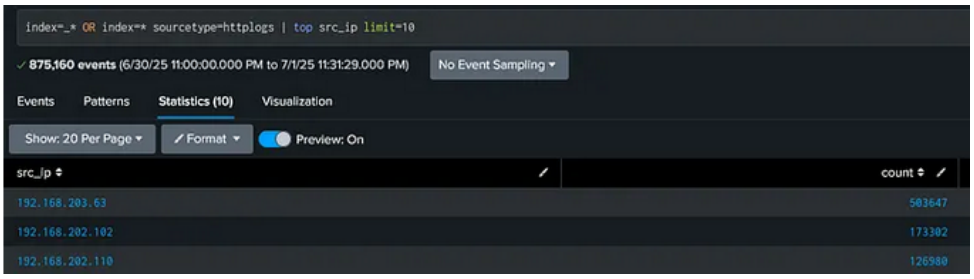

Figure 29: Top 10 source IPs by request count

13. Monitor User Behaviour

13.1 Failed Login Attempts

Identify source IPs with multiple failed login or unauthorised access attempts, which may indicate a brute-force or credential-stuffing attack:

```
index=_* OR index=* sourcetype=httplogs
| search action="login" status="failed"
| stats count by src_ip
| sort -count
```

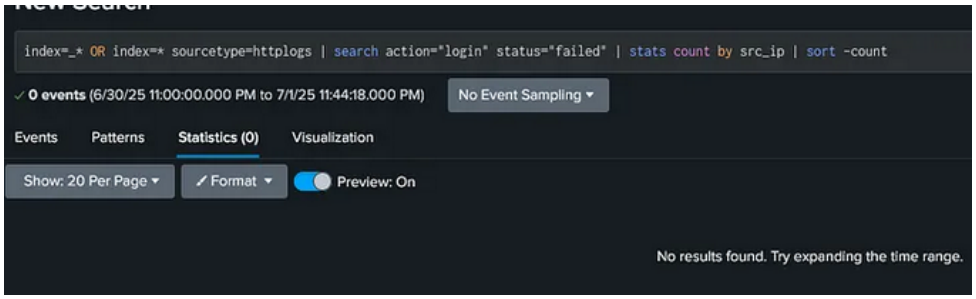

Figure 30: Source IPs with the highest count of failed login attempts

13.2 User Session Duration Analysis

Analyse session durations and access patterns to spot abnormal user behaviour, such as unusually long or short sessions:

```
index=_* OR index=* sourcetype=httplogs
| stats range(_time) as session_duration by session_id
| stats avg(session_duration) as avg_session_duration by user
```

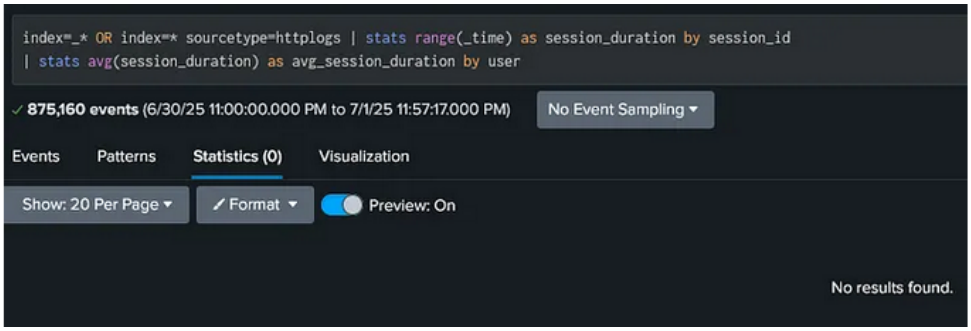

Figure 31: Average session duration per user derived from HTTP logs

Conclusion

Analyzing HTTP log files within the Splunk SIEM environment gives organizations comprehensive visibility into web server activity and client behaviour across the network. By monitoring HTTP events, identifying anomalous response patterns, and correlating findings with other log sources, security teams can strengthen incident detection capabilities and proactively defend against potential cyber threats.

> **Note:** The procedures and SPL queries demonstrated in this project can be tailored to align with specific operational environments, compliance frameworks, and organizational security requirements.

Technique	**Purpose**
Field Extraction (`rex`)	Parse structured fields from raw log text for richer searching and correlation.
`stats count by`	Aggregate events by a field to reveal distributions (methods, status codes, IPs).
`timechart`	Visualise event volume over time to detect traffic spikes or anomalies.
`where status >= 400`	Filter for HTTP error responses to focus on potential attacks or misconfigurations.
`top src_ip`	Rank source IPs by activity to surface suspicious hosts.
Session Analysis	Compute average session durations to identify abnormal user behaviour patterns.

Original post by Hassen Hannachi on Medium:
https://hassen-hannachi.medium.com/analyzing-http-log-files-using-splunk-siem-aabe342e1614